Apuntes del Sprint 11 · Visión por computador: cómo «ve» una red

ANTES DE EMPEZAR

este sprint se apoya en el Sprint 10 (redes, capas, Keras, sobreajuste, regularización). Si no recuerdas qué es una capa densa o un callback, repásalo.

Las diapositivas del Sprint 11 recorren **CNN, transfer learning, detección y segmentación** muy deprisa. Estos apuntes **explican cada idea con cuentas pequeñas que puedes hacer a mano en un papel**. Léelos antes de cada cuaderno.

0. De qué va el sprint (en 1 minuto)

TurisData recibe **cientos de fotos de alojamientos cada semana** y las clasifica **una persona a mano**. ¿Puede un modelo **pre-clasificarlas**?

Presentación	Pregunta	Cuaderno
S11.1 CNN	¿Cómo «ve» una red una imagen?	S11_01
S11.2 Transfer learning	¿Y si solo tengo 30 fotos?	S11_02
S11.3 Detección y segmentación	¿Qué hay, dónde, y cuándo **no** fiarse del modelo?	S11_03, S11_04 (reto)

El reto: un **demostrador** que recibe una foto y devuelve **clase, confianza y «revisar a mano» si la confianza es baja**, con análisis de errores.

> **Lo importante no es el acierto, sino saber cuándo no fiarse.** Un modelo que acierta el 90 % pero **no sabe cuándo falla** es peligroso.

DÍA 1 · Redes convolucionales (CNN)

1. Una imagen es una matriz de números

Tipo	Forma	Ejemplo
Escala de grises	`(alto, ancho)`	8×8 píxeles, de 0 (negro) a 16 (blanco) — los dígitos de scikit-learn
Color	`(alto, ancho, 3)`	3 **canales**: rojo, verde y azul
Un lote	`(n, alto, ancho, canales)`	n imágenes juntas

Para la red se **normaliza** (valores entre 0 y 1) y se añade el **canal explícito** (Keras lo espera):

```
X = (imagenes / 16.0)[..., np.newaxis]       # forma (n, 8, 8, 1), valores entre 0 y 1
```

2. La convolución: un filtro que se desliza

Un **filtro** (*kernel*) es una **mini-tabla**, por ejemplo de 3×3. Se coloca sobre una zona de la imagen, **se multiplica casilla a casilla, se suma** y sale **un número**. Luego se desliza y se repite. El resultado es un **mapa de rasgos**: **dónde aparece ese patrón** (un borde, un trazo).

Ejemplo con números: detectar un borde vertical

Imagen de 3 filas × 5 columnas (negro a la izquierda, claro a la derecha):

```
0 0 9 9 9
0 0 9 9 9
0 0 9 9 9
```

Filtro 3×3 (**«derecha menos izquierda»**):

```
-1 0 1
-1 0 1
-1 0 1
```

Se desliza por las 3 posiciones horizontales posibles:

Zona de la imagen (columnas)	Cálculo (derecha − izquierda, × 3 filas)	Resultado
Columnas 0-2 → `0 0 9`	(9 − 0) · 3	**27**
Columnas 1-3 → `0 9 9`	(9 − 0) · 3	**27**
Columnas 2-4 → `9 9 9`	(9 − 9) · 3	**0**

Mapa de rasgos: `[27, 27, 0]`. **Valores altos = hay borde vertical ahí; 0 = zona uniforme.** Una red tiene **muchos filtros** y **aprende** cuáles le sirven (bordes, esquinas, texturas).

¿De qué tamaño sale el mapa?

```
def tam_salida(n, k, p=0, s=1):
    return (n - k + 2 * p) // s + 1       # n: tamaño, k: filtro, p: relleno, s: paso
```

Con 8 píxeles y un filtro de 3 **sin relleno**: (8 − 3)/1 + 1 = **6**. Con `padding="same"` (relleno de 1): (8 − 3 + 2)/1 + 1 = **8** (se conserva el tamaño).

3. Pooling: quedarse con lo importante

Max pooling 2×2: se divide el mapa en bloques de 2×2 y **se queda con el máximo de cada bloque**.

```
1 3 | 2 0                →      4  5
4 2 | 1 5                       3  7
----+----
0 1 | 7 2
3 2 | 0 4
```

Máximos: (1,3,4,2) → **4**; (2,0,1,5) → **5**; (0,1,3,2) → **3**; (7,2,0,4) → **7**.

- Los mapas quedan **la mitad de grandes** → **menos cálculo**.
- **Tolerancia a pequeños desplazamientos:** si el trazo se mueve un píxel, el máximo del bloque suele ser el mismo.

```
def max_pooling(mapa, t=2):
    a = mapa.shape[0] // t
    return mapa.reshape(a, t, a, t).max(axis=(1, 3))
```

> **Pregunta del cuaderno:** ¿por qué una CNN necesita **muchos menos parámetros** que un MLP en imágenes grandes? **Porque reutiliza el mismo filtro en toda la imagen.** Ejemplo: una imagen de 224×224×3 aplanada tiene 150.528 entradas; con 100 neuronas, **una sola capa densa tendría más de 15 millones de pesos**. Una capa convolucional con 32 filtros de 3×3 tiene **solo 896** (3·3·3·32 + 32).

4. Una CNN completa en Keras

```
[Conv → ReLU → Pooling] × varias veces → Flatten → Dense → salida
```

Las **primeras capas** detectan **trazos simples**; las **siguientes**, **formas cada vez más complejas** (de bordes a curvas, de curvas a «ojos» o «ruedas»).

```
def construir_cnn(filtros=(16, 32)):
    m = keras.Sequential([keras.layers.Input((8, 8, 1))])
    for f in filtros:
        m.add(keras.layers.Conv2D(f, 3, padding="same", activation="relu"))
        m.add(keras.layers.MaxPooling2D(2))
    m.add(keras.layers.Flatten())
    m.add(keras.layers.Dense(10, activation="softmax"))
    m.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"])
    return m
```

Cómo leerlo, capa a capa:

Capa	Forma de salida	Qué hace
Entrada	8 × 8 × 1	La imagen
Conv2D (16 filtros) + pooling	4 × 4 × 16	16 mapas de rasgos, a la mitad de tamaño
Conv2D (32 filtros) + pooling	2 × 2 × 32	32 mapas más abstractos
Flatten	128	Estira todo en una fila (2·2·32)
Dense (10, softmax)	10	**Una probabilidad por cada dígito (0-9)**, suman 1

- **`softmax`**: convierte 10 números en 10 probabilidades que **suman 1** (en vez de la sigmoide, que sirve para una sola probabilidad).
- **`sparse_categorical_crossentropy`**: la pérdida para clasificar **varias clases** cuando la etiqueta es un número (0-9).
- Parámetros totales: **6.090** (160 + 4.640 + 1.290): muy pocos para lo que hace.

Mirar dentro de la red

- Los **filtros de la primera capa** son **pequeñas plantillas de trazos**.
- Los **mapas de activación** muestran **dónde se activa cada filtro** con un dígito concreto.

Es la forma de responder **«¿qué ha aprendido la red?»** sin tratarla como una caja negra.

5. Data augmentation

Si el alojamiento aparece **más a la izquierda, más pequeño o ligeramente girado**, **sigue siendo el mismo alojamiento**. El *data augmentation* **genera variaciones aleatorias** de las imágenes de entrenamiento (desplazar, girar, hacer zoom):

- Se aplica **solo al entrenar** (no al evaluar).
- **Reduce el sobreajuste** cuando hay pocas imágenes: la red ve «más» fotos distintas.

> **Pregunta de reflexión:** ¿en qué casos sería **un error** aplicar un giro aleatorio? **Pista: dígitos «6» y «9».** Si giras un 6 media vuelta, **¡se convierte en un 9!** Y con letras «b» y «d», o con una flecha de dirección, ocurre igual. **El aumento solo es válido si la etiqueta no cambia.**

DÍA 2 · Transfer learning y fine-tuning

6. El problema: pocas imágenes

TurisData solo tiene **6 imágenes por tipo de alojamiento** (30 en total). **Es muy poco para una CNN desde cero**: memoriza y rinde mal en prueba. **La solución: no empezar de cero.**

7. Base y cabeza

	Base	Cabeza
Qué es	Bloques **Conv → Pool** que **extraen rasgos** (terminan en `Flatten`)	La capa **`Dense`** que **decide la clase**
Qué aprende	**Bordes, texturas** (sirve para **muchas tareas**)	Algo **específico de tu problema**

Analogía: un **traductor** que ya sabe leer español e inglés no empieza de cero para aprender a traducir textos de turismo: **reutiliza lo que ya sabe** y solo se especializa en el vocabulario del sector.

> **En la vida real** se descarga una base entrenada con millones de fotos (**ImageNet**). **En el cuaderno**, para trabajar sin internet, la base se preentrena en una tarea auxiliar (reconocer 8 clases de formas y texturas). Los alojamientos son **dibujos esquemáticos**,

> no fotos reales.

8. Congelar la base y entrenar solo la cabeza

```
def congelar(modelo):
    modelo.trainable = False          # los pesos NO se actualizan
    return modelo
```

Se pone una **cabeza nueva** y **solo esta se entrena**. Ventajas: **se conserva lo aprendido**, y hacen falta **menos datos y menos tiempo**. Siempre se compara con **la CNN desde cero** como referencia: *¿cuánto aporta realmente el transfer learning?*

9. Fine-tuning: descongelar con cuidado

1. **Entrena primero la cabeza** con la base congelada.
2. **Descongela el último bloque** de la base (`base.trainable = True` y deja congeladas las capas anteriores por nombre).
3. **Sigue entrenando con una tasa de aprendizaje muy pequeña** (10 veces menor o más) para **ajustar rasgos específicos sin destruir lo aprendido**.

> **Tasa grande en el fine-tuning = la red «olvida» lo bueno que traía.** Es como corregir un cuadro terminado con un brochazo gigante.

10. Mini-reto: ¿cuántas imágenes necesito?

El cliente pregunta: *«¿cuántas fotos por tipo de alojamiento tenemos que etiquetar?»*. Se responde **con datos**:

```
for n in [3, 6, 12, 24]:                                # imágenes por clase
    X, y = generar_lote(dibujar_alojamiento, clases, n)
    entrenar(cnn_desde_cero, epocas, semilla, X, y)      # referencia
    entrenar(transfer_congelado, epocas, semilla, X, y)  # base preentrenada congelada
```

Gráfica de **precisión en prueba frente a n° de imágenes** para los dos modelos. Lo normal: **con muy pocas imágenes, el transfer learning gana por mucho; con muchas, la diferencia se reduce**. Tu respuesta debe decir **dónde deja de compensar etiquetar más** (por ejemplo, *«a partir de 12 por clase la mejora es solo de 1 punto»*).

DÍA 3 · Detección, segmentación y cuándo no fiarse

11. Tres tareas con imágenes

Tarea	Pregunta	Salida	Ejemplo TurisData
Clasificación	¿Qué hay en la foto?	Una **etiqueta**	«esta foto es de una piscina»
Detección	¿**Qué hay y dónde**?	**Cajas** `(x1, y1, x2, y2)` + clase + confianza	**Contar sombrillas** en la playa
Segmentación	¿**Qué píxeles** son?	Una **máscara**	**Superficie de arena libre**

12. Cajas, IoU y NMS

IoU: cuánto se solapan dos cajas

```
IoU = área de la intersección / área de la unión      (de 0 a 1)
```

Ejemplo: caja A = (0, 0, 4, 4) → área 16. Caja B = (2, 2, 6, 6) → área 16.

```
Intersección: de x=2 a 4 (ancho 2) y de y=2 a 4 (alto 2) → área 2·2 = 4
Unión = 16 + 16 − 4 = 28
IoU = 4 / 28 = 0,14      → casi no se solapan
```

Con B = (1, 0, 5, 4): intersección = 3·4 = 12; unión = 16 + 16 − 12 = 20; IoU = **0,60**. **Una detección es correcta si IoU ≥ 0,5** con la caja real.

NMS: quitar cajas repetidas

El detector propone **varias cajas casi iguales** para el mismo objeto. **NMS (supresión de no-máximos):**

1. **Ordena por confianza.**
2. **Quédate con la primera.**
3. **Descarta las que se solapan con ella más del umbral** (IoU > 0,5).
4. **Repite** con las que quedan.

Ejemplo: caja A (confianza 0,9), caja B (0,8, IoU con A = 0,6), caja C (0,7, lejos de A). Orden: A, B, C. **Se queda A**; B se solapa 0,6 > 0,5 con A → **se descarta**; C no se solapa → **se queda**. Resultado: A y C (2 objetos, no 3).

13. Métricas de detección: precisión, recall, AP y mAP

Ejemplo: hay **4 sombrillas reales** y el detector da **5 cajas**, ordenadas por confianza (= acierta con IoU ≥ 0,5, = falso positivo):

Puesto	Confianza	¿Correcta?	Precisión acumulada	Recall acumulado
1	0,95		1/1 = 1,00	1/4 = 0,25
2	0,90		2/2 = 1,00	2/4 = 0,50
3	0,80		2/3 = 0,67	2/4 = 0,50
4	0,70		3/4 = 0,75	3/4 = 0,75
5	0,60		3/5 = 0,60	3/4 = 0,75

- **Precisión acumulada** = aciertos hasta aquí / cajas dadas hasta aquí.
- **Recall acumulado** = aciertos hasta aquí / sombrillas reales (4).

AP (*average precision*) = **área bajo la curva precisión-recall**, con la precisión **«suavizada» de derecha a izquierda** (cada valor pasa a ser el máximo de los que tiene a su derecha: 1,00 · 1,00 · 0,75 · 0,75 · 0,60):

```
AP = 0,25·1,00 + 0,25·1,00 + 0,25·0,75 = 0,25 + 0,25 + 0,1875 = 0,6875
```

(Cada término = *aumento de recall × precisión suavizada*. Del recall 0,5 al 0,75 la precisión suavizada es 0,75; del 0,75 en adelante no hay más aumento.)

mAP@[.5:.9]: la **media del AP** con umbrales de IoU de 0,5 a 0,9. Exige cajas cada vez **más ajustadas**.

14. Un detector clásico y la segmentación

En el cuaderno se genera una **escena sintética de playa** (128×128): arena, mar, **sombrillas** (círculos de colores, a veces solapados) y **distractores**. El **`detector_color` con OpenCV** (sin red neuronal): **umbral de color + contornos → cajas**. `evaluar_detector` da precisión, recall y AP.

> **No todo requiere deep learning:** cuando el objeto es muy distinguible (un color único), **un detector clásico es rápido, barato y explicable**.

Segmentación: una máscara por píxel

Se mide con la **IoU de máscaras** o con el **Dice**:

```
iou_mascara = (pred & real).sum() / (pred | real).sum()
dice = 2 * (pred & real).sum() / (pred.sum() + real.sum())      # 1.0 si ambas están vacías
```

Ejemplo: la máscara real tiene 6 píxeles; la predicha, 4; **coinciden en 3**.

```
IoU  = 3 / (4 + 6 − 3) = 3/7 = 0,43
Dice = 2·3 / (4 + 6)  = 6/10 = 0,60
```

El **Dice siempre sale algo mayor** que la IoU para el mismo caso: no los compares entre sí, **elige uno y úsalo siempre**.

15. Del cuaderno al reto: cuándo no fiarse

Pieza (cuaderno S11_04)	Función	Para qué
Dataset por carpetas	`contar_por_clase(ruta)`	Ver si las clases están equilibradas
Transfer learning	`modelo_transfer(base, n_clases)`	El modelo
Matriz de confusión	`recall_clase`, `precision_clase`	Métricas **por clase**
Análisis de errores	`indices_errores(y_real, prob)`	Ver **las fotos fallidas**, de mayor a menor confianza
El modelo pide ayuda	**umbral de confianza**	Cobertura y precisión sobre lo aceptado

Análisis de errores: la parte más valiosa

No basta con decir «acierta el 90 %». Hay que **mirar las fotos mal clasificadas** y buscar el patrón: *¿fotos oscuras? ¿ángulos raros? ¿dos clases que se parecen?* **Con una hipótesis de por qué** (el reto pide analizar **al menos 6 fotos**).

Umbral de confianza: «si dudas, pregunta»

El demostrador acepta la predicción **solo si la confianza supera un umbral**; si no, responde **«revisar a mano»**.

Ejemplo: 100 fotos y un umbral de 0,80. El modelo **acepta 70** y acierta **66**; las otras 30 pasan a una persona.

```
Cobertura  = 70 / 100 = 70 %    (lo que resuelve solo)
Precisión sobre lo aceptado = 66 / 70 = 94,3 %
```

Subir el umbral → menos cobertura (más trabajo humano) pero **más precisión**. **Bajarlo** → más automático, **más errores**. Es un **equilibrio de negocio** (como el umbral del Sprint 7), y se elige según el coste de un error.

16. El demostrador del reto

1. **Preparar el conjunto** (el sintético o uno propio **con licencia**): **una carpeta por clase**, sin espacios ni tildes, **mínimo 30 imágenes por clase**.
2. **Transfer learning + referencia desde cero.**
3. **Evaluar en prueba** y **analizar al menos 6 fotos mal clasificadas**, con una hipótesis de por qué.
4. **Demostrador:** una función que recibe una foto y devuelve **clase, confianza y «revisar a mano»** si la confianza es baja.

> **Cuidado con las imágenes:** usa solo imágenes **con licencia o propias**, y revisa que no aparezcan **personas identificables** (RGPD, Sprint 1).

17. Dónde practicar cada cosa

Si no entiendes...	Mira la sección	Practica en
Imagen como matriz, convolución, pooling, CNN, augmentation	1-5	`S11_01_cnn`
Base y cabeza, congelar, fine-tuning	6-10	`S11_02_transfer_learning`
IoU, NMS, AP, segmentación, detector con OpenCV	11-14	`S11_03_deteccion_segmentacion`
Análisis de errores, umbral de confianza, demostrador	15-16	`S11_04_reto_vision_cliente`

18. Chuleta de vocabulario

Palabra	En lengua normal
Píxel	Cada número de la imagen
Canal	Cada capa de color (rojo, verde, azul)
Filtro / kernel	Mini-tabla que se desliza buscando un patrón
Mapa de rasgos	Dónde aparece el patrón del filtro
Pooling	Reducir el mapa quedándose con lo más importante

Palabra	En lengua normal
CNN	Red con capas convolucionales
Softmax	Convierte números en probabilidades que suman 1
Data augmentation	Variar las imágenes al entrenar
Transfer learning	Reutilizar una red ya entrenada
Base / cabeza	Parte que extrae rasgos / parte que decide la clase
Congelar	Impedir que los pesos cambien
Fine-tuning	Ajustar la base con tasa muy pequeña
Detección	Qué hay y dónde (cajas)
Segmentación	Qué píxeles son (máscara)
IoU	Intersección / unión de dos cajas o máscaras
NMS	Quitar cajas repetidas
AP / mAP	Calidad de un detector (área precisión-recall)
Dice	Medida de solapamiento de máscaras
Umbral de confianza	Valor por debajo del cual se pide ayuda humana

19. Cómo estudiar estos apuntes

1. **Antes de cada cuaderno**, lee la sección que indica la tabla del apartado 17.
2. **Haz a mano** la convolución (sección 2), el IoU (sección 12) y el AP (sección 13): con 3 números en papel entenderás qué calcula el ordenador.
3. **Mira siempre las fotos que falla el modelo**: ahí está lo que aprendes.
4. **En el reto, la nota no es el acierto**, sino **demostrar que sabes cuándo no fiarte**: el «revisar a mano» es la mitad del producto.